

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Таблицы решений, state-transition, исследовательское тестирование, баг-репорты

Модуль 2 · Лекция 4 · Тест-дизайн

Ольга Лихачева · Lектор

4 курс

24 сентября · Четверг 13:30–15:00

План лекции

- 01** Как выбрать технику под задачу
- 03** Диаграммы и таблицы состояний/переходов (state-transition)
- 05** Баг-репорты: структура, severity vs priority, реальный пример
- 07** Практика 2 (25.09): peer review и сдача артефакта

- 02** Таблицы решений (decision tables) — сквозной пример на клиенте
- 04** Исследовательское тестирование (exploratory testing)
- 06** LLM в тест-дизайне: условия применимости и ограничения

Разогрев: что мы уже умеем

Лекция 3 — база

Классы эквивалентности: какие значения система обрабатывает одинаково.

Граничные значения: что происходит на стыке классов.

Попарное тестирование: как покрыть комбинации параметров.

Лекция 4 — что добавим

Таблицы решений: когда решение зависит от комбинации условий.

State-transition: когда поведение зависит от состояния и порядка.

Исследовательское тестирование: когда документации мало.

Баг-репорты: как зафиксировать найденный дефект.

Вопросы в зал. Первый: что нельзя смешивать в одном тест-кейсе? (невалидные классы). Второй: сколько точек нужно на границу? (три). Третий: что делать, если требование молчит о формате поля? (спросить аналитика).

Выбор техники: когда какую

Ситуация	Техника
Любой объект тестирования (база)	Классы эквивалентности + граничные значения
Зависимости между условиями, бизнес-правила	Таблицы решений
Состояния влияют на поведение, важен порядок операций	State-transition
Много параметров с неизвестными зависимостями	Pairwise
Низкое качество документации, нет времени	Исследовательское тестирование

Сквозной пример лекции: заведение клиента

Один объект на всю лекцию: `POST /api/v1/clients/create-individual` — тот же запрос, что на лекции 3.

Поля запроса

`contacts.email, contacts.phone`
`individual.demographic.birth_date`
`individual.demographic.gender`
`individual.demographic.marital_status`
`tax.tax_number, external_id`

Что показывает каждая техника

Таблица решений — от чего зависит код ответа при создании.

State-transition — жизненный цикл клиента: создан, активен, заблокирован.

Exploratory — что исследовать в этом запросе.

Баг-репорт — как описать найденный дефект.

Так один объект связывает все четыре темы и практику 2, где вы примените техники к Authorization.

МОДУЛЬ 2 · ЛЕКЦИЯ 4

Таблицы решений

Таблицы решений: определение и структура

Таблица решений — компактная запись сложных бизнес-правил.

Условия — верх

Набор условий, от которых зависит решение.

Действия — низ

Что система должна сделать.

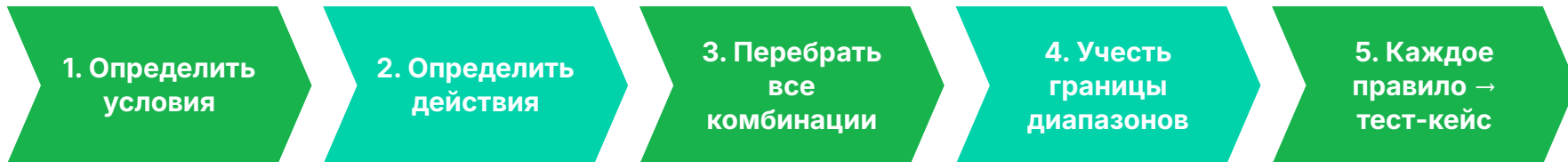
Правила — столбцы

Каждое правило = уникальная комбинация условий → набор действий.

Действия зависят **только от значений условий**, не от порядка.

Из таблицы напрямую получаются тест-кейсы.

Таблицы решений: методика построения



Комбинаторика

Для n бинарных условий — 2^n правил. Условия бывают бинарные или диапазонные.

Диапазонные условия

Учесть границы диапазонов — таблица решений комбинируется с BVA.

Ограничение: рост экспоненциальный. Больше пяти-шести условий — таблица становится громоздкой.

Таблицы решений: пример 1 — автострахование

Условия: «состоит в браке?», «хороший студент?» (Да/Нет). Всего 4 правила.

(Да, Да)

Скидка 60%

(Да, Нет)

Скидка 25%

(Нет, Да)

Скидка 50%

(Нет, Нет)

Скидка 0%

Каждое правило → тест-кейс с ожидаемым действием.

Таблицы решений: пример 2 — заказ на покупку

Условия: «верный идентификатор?», «верное количество?», «достаточно средств?» → 8 правил. Купить можно, только если все три — «Да».

Схлопывание

Значение «не важно» (NB): если ID неверный — остальные условия неважны, правило одно.

Для тестов

Схлопнутая таблица удобна для разработки, но для тест-кейсов берём **несхлопнутую**.

Оптимизация таблицы сама может внести ошибки — используйте полную таблицу как основу тестов.

Сквозной пример: таблица решений для клиента

Условия создания клиента и ожидаемый результат

Условие	Возможные значения
Обязательные поля заполнены	Да / Нет
Форматы полей корректны	Да / Нет
Возраст по birth_date ≥ 18	Да / Нет
external_id не дублируется	Да / Нет

Четыре бинарных условия $\rightarrow 2^4 = 16$ правил. Действие одно из трёх: 201 Created, 400 VALIDATION_ERROR, 409 DUPLICATE.

Для тестов берём **несхлопнутую** таблицу: даже если поле не заполнено, проверяем, как система реагирует на остальные условия.

Сквозной пример: разбор правил клиента

Обязат.	Форматы	≥ 18	external_id	Ожидаемый результат
Да	Да	Да	уникален	201 Created, клиент создан
Да	Да	Да	дубликат	409 DUPLICATE
Да	Да	Нет	уникален	400, код AGE_LIMIT
Да	Нет	Да	уникален	400, код VALIDATION_ERROR
Нет	—	—	уникален	400, REQUIRED_FIELD

Каждая строка — **тест-кейс** с ожидаемым кодом и телом ответа.

Последняя строка — пример **схлопывания**: без обязательных полей остальные условия неважны.

Таблицы решений: применение и ограничения

Применять

Сложные бизнес-правила.

Комбинации условий с дискретными действиями.

Решения, где важна полнота перебора.

Не применять

Система без правил-комбинаций — таблица вырождается.

Более шести-семи условий — таблица становится нечитаемой.

Поведение зависит от порядка — это state-transition.

СМП: таблица решений авторизации = статус карты × лимиты × баланс → код ответа.

МОДУЛЬ 2 · ЛЕКЦИЯ 4

State-transition

State-transition: термины



Состояние

Период ожидания события; система «помнит» прошлое



Переход

Изменение состояния по событию



Событие

Триггер перехода — внешнее или внутреннее (например, таймер)

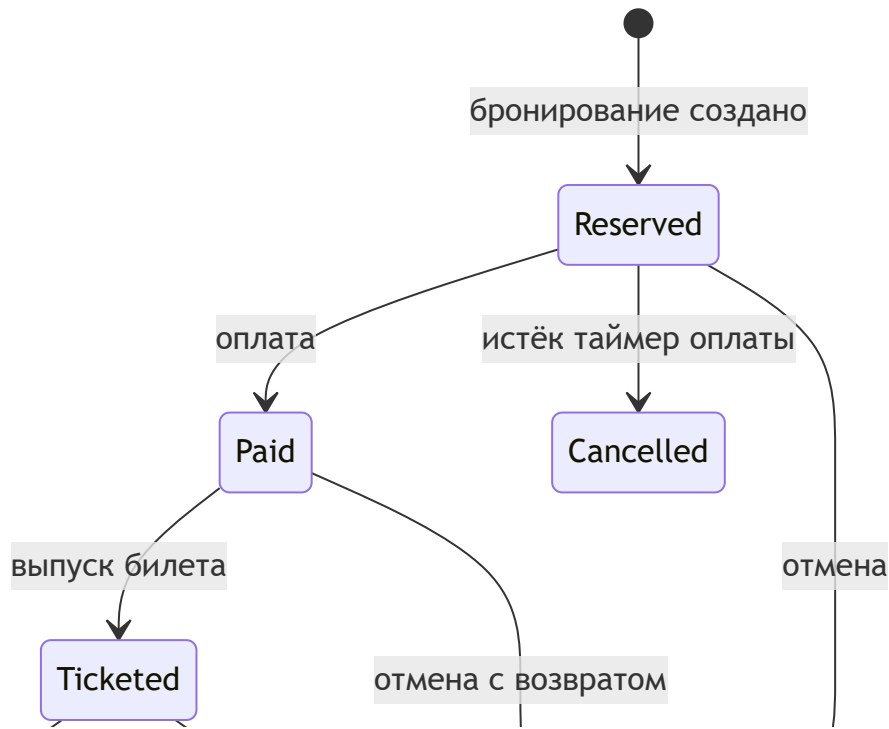


Действие

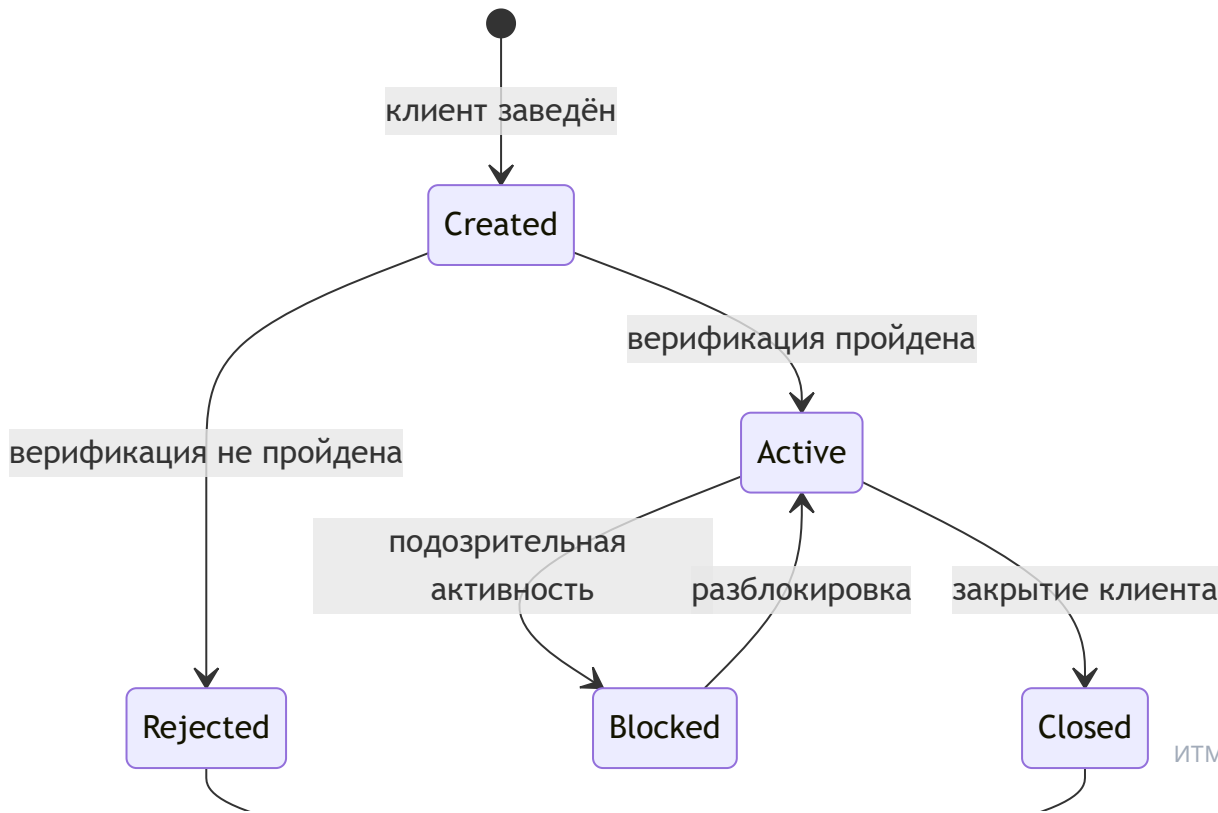
Операция, вызванная переходом — записывается после «/»

Нотация: чёрная точка — старт, «бычий глаз» — конечное состояние.

State-transition: диаграммы (пример бронирования)



Сквозной пример: жизненный цикл клиента



State-transition: таблицы переходов

Таблица из четырёх колонок: Текущее состояние | Событие | Действие | Следующее состояние.

Перечисляет **все** пары «состояние × событие», включая недопустимые.

Польза: помогает найти «дыры» в требованиях до написания кода.

Недостаток: таблица быстро растёт — много пустых клеток.

Пример: «Created + оплата» — недопустимая пара, о которой требования часто молчат.

Сквозной пример: таблица переходов клиента

Текущее состояние	Событие	Следующее состояние	Ожидание
Created	верификация пройдена	Active	разрешено
Created	открыть счёт	Created	отказ, код NOT_VERIFIED
Blocked	разблокировать	Active	разрешено
Active	разблокировать	Active	отказ, код INVALID_STATE
Closed	любое	Closed	отказ, код CLIENT_CLOSED

Строки 2, 4 и 5 — **недопустимые переходы**. Каждая — негативный тест-кейс, который часто отсутствует в требованиях.

State-transition: уровни покрытия

1

Уровень 1 — все состояния

Каждое состояние посещено хотя бы раз (низкий уровень)

2

Уровень 2 — все события

Каждое событие выполнено хотя бы раз

3

Уровень 3 — все пути

Может быть бесконечно из-за петель

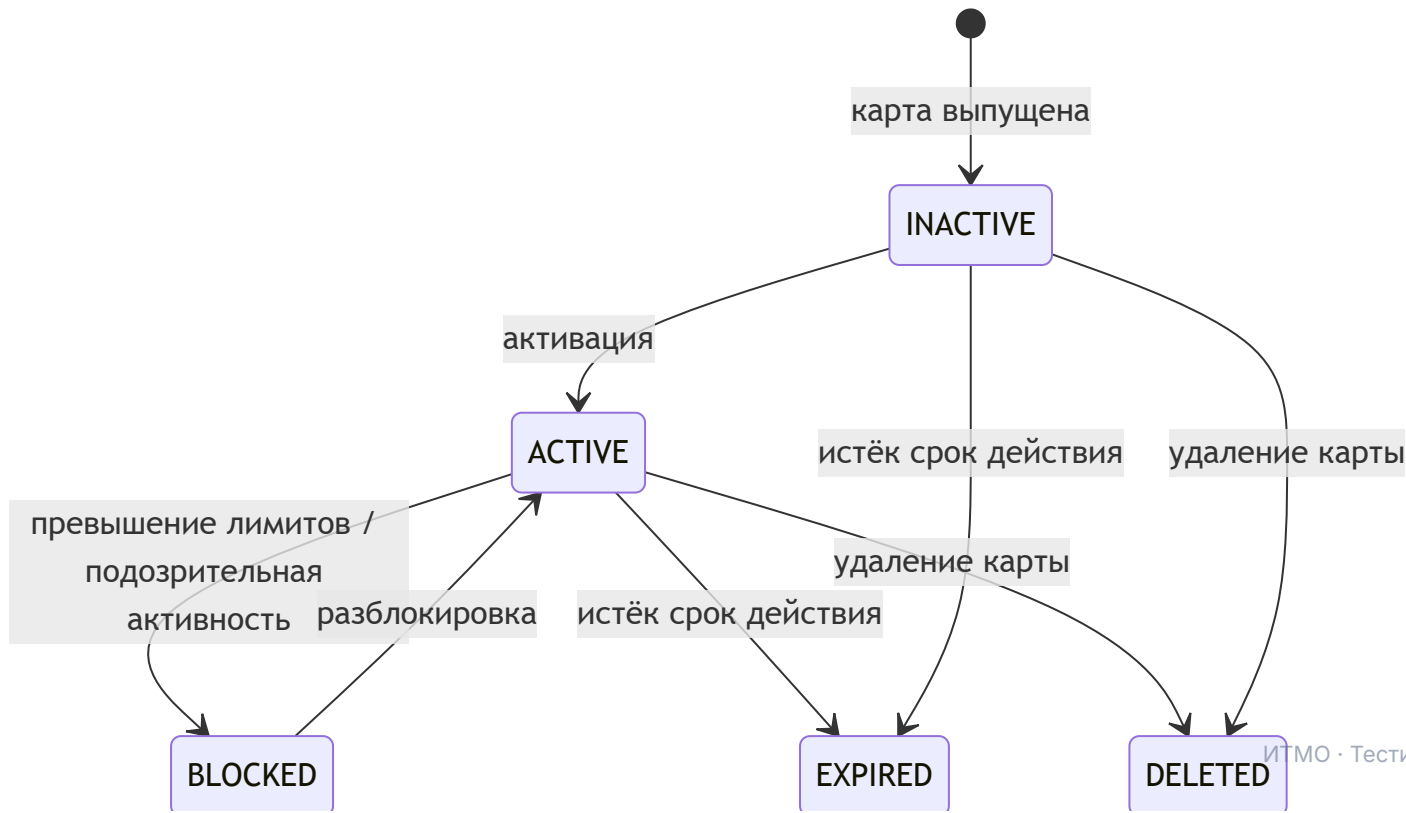
4

Уровень 4 — все переходы

Рекомендуемый уровень покрытия

Для критичных систем дополнительно проверяем **недопустимые переходы**.

State-transition в СМП: статусы карты



State-transition: применение и ограничения

Применять

Поведение зависит от состояния и порядка операций.

Есть жизненный цикл объекта: клиент, карта, заказ.

Нужно найти дыры в требованиях.

Не применять

Поведение не зависит от предыстории.

Результат определяется только комбинацией условий — это таблица решений.

Состояний и событий слишком много.

Правило выбора: если порядок операций меняет результат — state-transition; если порядок не важен, а важна комбинация — таблица решений.

МОДУЛЬ 2 · ЛЕКЦИЯ 4

Исследовательское тестирование

Исследовательское тестирование: определение

Классическое определение (Kaner): ЕТ — стиль тестирования, где **изучение, планирование и выполнение проверок идут параллельно** и взаимообогащают друг друга.

В каждом тесте есть исследовательская компонента; ЕТ — это «осознанное» исследование, а не «кликание».

Testing vs Checking: машины проверяют (checking), люди тестируют (testing).

ЕТ ≠ «беспорядочное» тестирование: структуру задают цели, чартеры и время.

Категория ISTQB: **experience-based** — техники, основанные на опыте тестировщика.

Исследовательское тестирование: когда применять и мифы

Когда применять

Плохая или отсутствующая документация.
Мало времени.
Подозрение на слабые тест-кейсы.
Сторонние компоненты, верификация фиксов.

Мифы

X «ET — unstructured» → структурируется целями, чартерами, временем.
X «ET = random» → выбор следующего шага основан на знании продукта.

Не использовать как первую и единственную технику — ET дополняет систематические техники.
Результаты ET плохо поддаются измерению — фиксируйте их через отчёты.

Сквозной пример: чартер для заведения клиента

Чартер: «Исследовать `POST /api/v1/clients/create-individual` на предмет поведения системы при частично заполненных и структурно некорректных данных, за 60 минут».

Вопросы исследования

Что будет, если передать `null` вместо отсутствующего поля?
Что если `contacts` — пустой массив?
Что если два поля нарушают правила одновременно?
Различаются ли сообщения об ошибке для разных нарушений?
Что происходит при повторном запросе с тем же `external_id` сразу?

Модели для исследования (Hendrickson)

Данные: границы, пустоты, спецсимволы.
Состояния: клиент уже существует, дубликат.
Взаимодействия: порядок полей, вложенность.
Окружение: повторный запрос, параллельность.

Исследовательское тестирование: практика



Test charter

«Почему? Что? Как? Какие проблемы?» — 1–2 часа на чартер



Session-based testing

Сессии около 90 минут: цель, время, отчёт



Эвристики и чек-листы

Hendrickson, Test Heuristics Cheat Sheet — источник идей для проверок



Модели для исследования

Состояния, данные, взаимодействия (Hendrickson, «Explore It!»)



МОДУЛЬ 2 · ЛЕКЦИЯ 4

Баг-репорты

Баг-репорты: зачем и кому

Баг-репорт — **основной видимый результат работы тестировщика**: по нему судят о его квалификации.

Читатели: разработчик (воспроизвести и найти причину), тест-менеджер, руководство.

Качественный отчёт = «продажа» исправления; плохой — повод отложить или проигнорировать.

Баг-репорт — заявление, что продукт стал **менее ценным для кого-то** (Kaner).

Сегодня — теория. Практика по баг-репортам — модуль 7 (E2E), по итогам реального прогона сценариев.

Баг-репорты: структура отчёта

Обязательное ядро

Заголовок (что, где, при каких условиях)

Шаги воспроизведения (Steps to Reproduce)

Ожидаемый результат

Фактический результат

Специфичность STR: детали важнее — иначе не воспроизвести

Типовые атрибуты (STF Table 6-1)

ID, объект теста, версия, платформа, дата, статус

Severity, priority

Связанное требование, тест-кейс

Источник проблемы, комментарии, ссылки

Вложения: скриншоты/видео, файлы; поля Jira: Summary, Priority, Environment, Description, STR

Сквозной пример: плохой баг-репорт

Заголовок: «Не работает создание клиента»

Шаги воспроизведения	1. Отправить запрос на создание клиента 2. Получить ошибку
Ожидаемый результат	Клиент создан
Фактический результат	Ошибка
Severity / Priority	—

Нет данных запроса: какие поля, какие значения.

Не указан код и тело ответа.

Не указаны severity и priority.

Сквозной пример: хороший баг-репорт

BR-001: Создание клиента возвращает 201 при email длиной 255 символов вместо 400 VALIDATION_ERROR

Шаги воспроизведения	1. POST /api/v1/clients/create-individual 2. Заполнить contacts.email строкой из 255 символов (см. вложение) 3. Остальные поля — валидные значения (см. вложение)
Ожидаемый результат	400, код VALIDATION_ERROR, клиент не создан
Фактический результат	201 Created, клиент создан
Связанное требование	email длиной не более 254 символов
Severity / Priority	SERIOUS (данные вне контракта попадают в систему) / NEXT RELEASE

Реальный пример из практики

Место для реального примера. Здесь на занятии разбирается действительный баг-репорт из рабочей практики: заголовок, шаги воспроизведения, ожидаемый и фактический результат, severity и priority.

На что смотрим в реальном примере

Как сформулирован заголовок.

Насколько специфичны шаги.

Как обоснованы severity и priority.

Какие атрибуты используются в реальном трекере.

Что обсудить со студентами

Почему дефект классифицирован именно так.

Что помогло воспроизвести проблему.

Как отчёт повлиял на приоритет исправления.

Что бы вы улучшили в формулировках.

Баг-репорты: severity vs priority

Severity (важность)

Степень ущерба от дефекта — с точки зрения пользователя.

Не меняется без новых данных о дефекте.

Классы (STF): 1 FATAL, 2 VERY SERIOUS, 3 SERIOUS, 4 MODERATE, 5 MILD.

Priority (срочность)

Как быстро исправить — зависит от контекста проекта.

Может меняться: договорённости, релиз, заказчик.

Классы (STF): 1 IMMEDIATE, 2 NEXT RELEASE, 3 ON OCCASION, 4 OPEN.

Severity $\neq$ Priority: серьёзный дефект может иметь низкий приоритет (и наоборот — косметический с высоким). Пример: ошибка расчёта дат до 1999 г. против перевёрнутого логотипа.

Баг-репорты: типичные ошибки и стандарты

Типичные ошибки

Расплывчатый заголовок.

Отсутствие важных шагов или лишние шаги.

«Решение вместо проблемы».

Несколько дефектов в одном отчёте.

Стандарты классификации

IEEE 1044-2009 — классификация аномалий (формально отозван в 2020).

Актуальная линейка — ISO/IEC/IEEE 29119; шаблоны документации — IEEE 829 / 29119-3.

На практике шаблон задаёт тест-менеджер единым для проекта.

Единый шаблон баг-репорта курса

Поле	Что указывать
ID	Уникальный идентификатор, например BR-001
Заголовок	Что, где и при каком условии — одной строкой
Шаги воспроизведения	Нумерованные, со конкретными данными и endpoint
Ожидаемый результат	Код ответа, тело, состояние системы
Фактический результат	Что произошло в действительности
Severity	Степень ущерба по шкале
Priority	Срочность исправления по шкале
Связанное требование	Требование или тест-кейс, который нарушен

LLM в тест-дизайне: условия применимости и ограничения

Ключевая мысль: LLM при одном и том же запросе даёт **разный набор тестовых сценариев** — результат не воспроизводится.

Тест-дизайн инженерен, потому что воспроизводим

Один вход — один структурированный выход с покрытием классов и границ.

LLM недетерминирован

Один промпт — разные наборы; результат нельзя повторить, проверить и защитить.

Отдельный блок слайдов с примерами — **«LLM в тест-дизайне: условия применимости и ограничения»**. Разбираем условия применимости, порядок изучения новой темы и четыре ограничения.

Резюме лекции 4

Таблицы решений: правила = тест-кейсы; каждый столбец — комбинация условий; для тестов — несхлопнутая таблица.

State-transition: покрытие всех переходов; таблицы переходов ловят «дыры» в требованиях; применяется, когда важен порядок операций.

Исследовательское тестирование: изучение, планирование и проверка идут параллельно; чартеры и сессии; дополняет систематические техники.

Баг-репорт: заголовок + STR + expected + actual + severity + priority; практика — в модуле 7.

LLM: недетерминирован, не заменяет технику — только черновики и вопросы.

Практика 2 (25.09): сдача артефакта — стратегия, тестовый план, чек-листы, классы эквивалентности, границы и pairwise по Authorization / Card-Management; peer review.

Практика 2 · Проходная (25.09)

1. Peer review тестовой документации по шаблону артефакта
2. Сдача артефакта 2 четырьмя документами: стратегия, план, test-design, чек-листы
3. Test-design: классы эквивалентности, границы, pairwise по Authorization / Card-Management
4. Модель PICT и попарный набор в docs/practice-2/pict/
5. Проверка по LLM-рубрике (skill-1) и peer review
6. Дедлайн сдачи артефакта 2 – 25.09

Артефакт 2 · 7 баллов · баллы начисляются только при наличии всех компонентов

Источники

ISTQB — Certified Tester Foundation Level (CTFL) v4.0, Syllabus и Glossary.

Copeland, Lee. *A Practitioner's Guide to Software Test Design*. Artech House.

Куликов, С. С. *Тестирование программного обеспечения*. Базовый курс.

Kaner, Bach, Pettichord. *Lessons Learned in Software Testing* (LLST / TTS).

Kaner. *A Tutorial in Exploratory Testing*.

Hendrickson, Elisabeth. *Explore It!*; Test Heuristics Cheat Sheet.

ISO/IEC/IEEE 29119 — документация и техники тестирования; исторический ориентир — IEEE 829.

IEEE 1044-2009 — классификация аномалий (переведён в статус неактивного в 2020).

The background is a dark blue gradient. It features several stylized, glowing green classical buildings with columns, resembling temples or banks. These buildings are interconnected by a network of glowing green lines that resemble circuitry or data flow. There are also several glowing green rectangular shapes, possibly representing tablets or screens, scattered throughout the scene.

Вопросы?